

Assignment 2: Principles in Practice – Microservices & Event-Driven Architecture

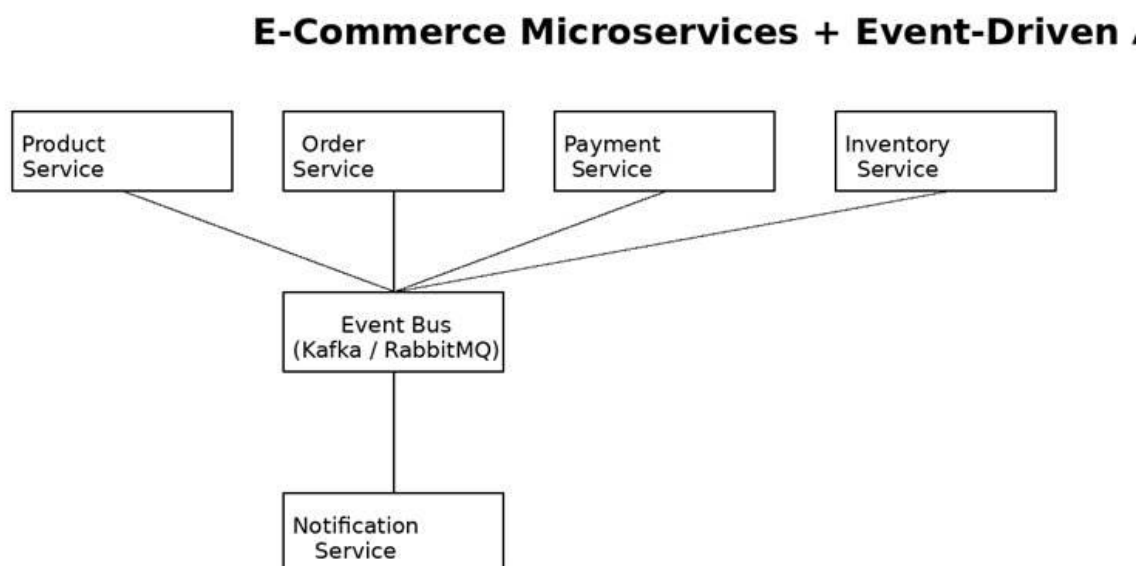
This scenario describes the design of a hypothetical e-commerce platform using Microservices Architecture combined with Event-Driven Architecture (EDA). The goal is to build a scalable, maintainable, and resilient system while applying SOLID, DRY, and KISS principles in practice.

Scenario Overview

The e-commerce platform supports product browsing, order placement, payment processing, inventory management, and order notifications. The system must handle high traffic, support independent feature updates, and remain responsive during failures.

Architecture Diagram: Microservices with Event-Driven Architecture

The following diagram illustrates a hypothetical e-commerce platform designed using Microservices Architecture combined with Event-Driven Architecture.



Explanation:

- Each business capability is implemented as an independent microservice.
- The Order Service publishes events to the Event Bus.
- Other services consume events asynchronously.
- This design ensures loose coupling, scalability, and fault tolerance.

Applying Microservices Architecture

The platform is decomposed into independent microservices, each responsible for a single business capability:

- Product Service – Manages product catalog and pricing.
- Order Service – Handles order creation and tracking.
- Payment Service – Processes payments securely.
- Inventory Service – Updates stock levels.
- Notification Service – Sends emails and messages to users.

Each microservice has its own database and can be developed, deployed, and scaled independently.

Applying Event-Driven Architecture (EDA)

An event-driven approach is used for communication between services. When a customer places an order, the Order Service publishes an 'OrderPlaced' event.

- Inventory Service listens to the event and updates stock.
- Payment Service processes the payment.
- Notification Service sends confirmation to the customer.

This loose coupling ensures services do not depend directly on each other, improving scalability and fault tolerance.

Applying SOLID Principles

SOLID principles improve code structure and maintainability in each microservice:

- Single Responsibility Principle (SRP): Each microservice and class handles one responsibility.
- Open/Closed Principle (OCP): Services can be extended with new features without modifying existing code.
- Liskov Substitution Principle (LSP): Service interfaces allow interchangeable implementations.
- Interface Segregation Principle (ISP): Services expose small, specific APIs.
- Dependency Inversion Principle (DIP): High-level services depend on abstractions, not concrete implementations.

Observing DRY and KISS Principles

DRY (Don't Repeat Yourself):

- Shared utilities (logging, authentication) are centralized.
- Common event schemas are reused across services.
- Duplicate business logic is avoided across microservices.

KISS (Keep It Simple, Stupid):

- Each microservice focuses on a single business function.
- Simple event formats are used for communication.
- Clear and minimal APIs reduce system complexity.

Conclusion

By combining Microservices Architecture with Event-Driven Architecture, the e-commerce platform achieves high scalability and resilience. Applying SOLID principles ensures clean and maintainable service design, while DRY and KISS principles help keep the system simple, reusable, and efficient.